PARALLEL

Your attention digital twin

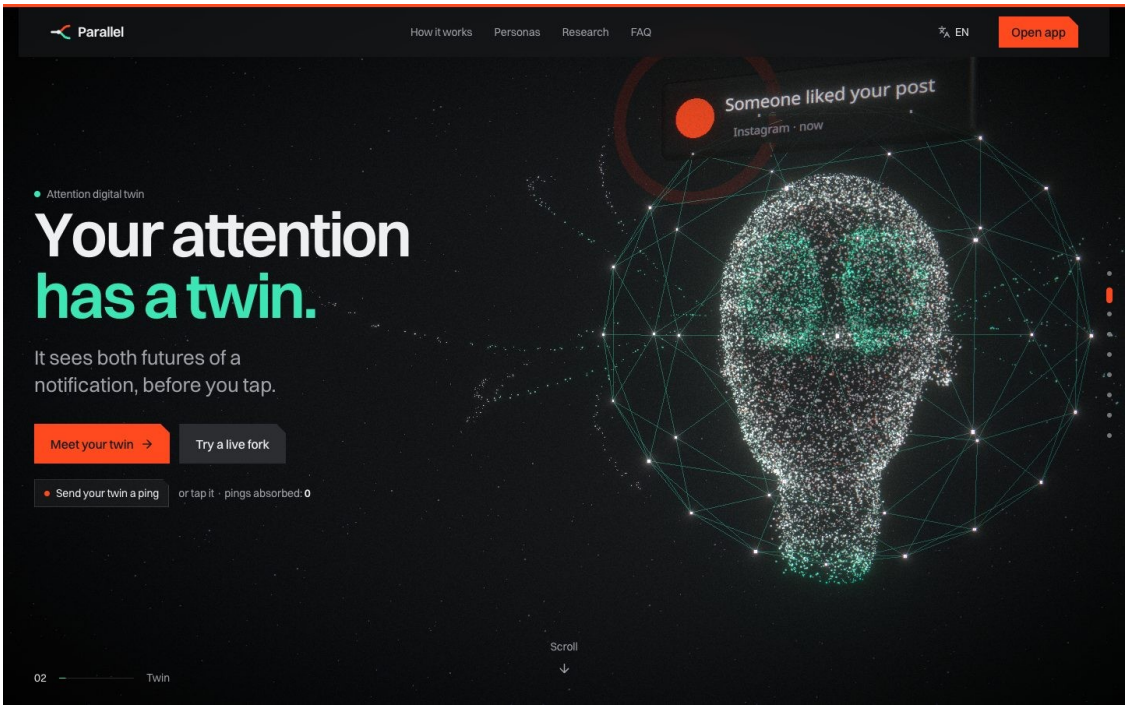
Every ping forks your day. Parallel shows you both futures before you tap.

Hackathon: CASCADE ’26 · Theme 3 · Problem 3: The Attention Digital Twin

Document: Project report: problem, gaps, solution, approach, architecture, use cases, feasibility, data and implementation

Code: github.com/SDP42/parallel

Date: October 2026



The landing page: your attention’s twin as a hologram head (live WebGL).

Contents

1. Executive summary.....
2. The problem.....
3. Who faces it.....
   What a “persona” means in Parallel.....
   Who faces the problem.....
4. Gaps in current solutions.....
5. Our solution.....
6. Our unique approach.....
7. Key features (with prototype screenshots).....
   7.1 Today: state, inputs and recommendations.....
   7.2 Multiverse simulator.....
   7.3 Smart inbox and auto-reply.....
   7.4 Attention fingerprint and Twin brain.....
   7.5 Time machine and doppelgänger.....
   7.6 Routine and schedule.....
   7.7 Data & models.....
   7.8 Mobile app.....
   7.9 The 3D landing story.....
8. Technology.....
9. System architecture.....
10. Use cases.....
11. Data inputs needed.....
12. Machine learning and evaluation.....
13. How to actually implement it.....
14. Feasibility.....
15. Risks and mitigations.....
16. Honest limits.....
17. Appendix: running the project.....

1. Executive summary

Parallel is a personal attention digital twin. It learns when you focus best, what pulls you away, which content turns quick checks into long sessions, which notifications actually matter to you, when you are running low, and which apps fragment your concentration.

It then answers the question at the heart of the problem statement: “If I allow this notification right now, what is likely to happen?” For each ping it simulates two futures. In Universe A you tap, and it shows the likely chain and the focus-minutes lost. In Universe B the twin holds the ping until your next natural break. Every estimate comes with a range and a confidence. A randomized hold-back policy makes the counterfactual honest, not just a correlation.

What is built: a working prototype with:

- a FastAPI backend with about 85 endpoints
- per-user twins trained from a SQLite database
- a population dataset of 81,582 notification events and population models evaluated on unseen users (p(open) AUC 0.845 vs a 0.728 baseline)
- a routine-aware decision engine and a 3D scroll-story landing page
- 12 app screens in four languages with a voice coach
- an installable PWA, plus Android and iOS projects from the same codebase

2. The problem

The problem statement asks for a twin that learns each person’s attention patterns and simulates the consequences of a notification, including what would have happened otherwise. Its example: a user is coding and receives “Someone liked your post”. The predicted outcome is a 2-minute interruption, a social app opened, 17 minutes of browsing and an 8-minute task-resumption delay, so the system can suppress the interruption.

The underlying problem is not screen time. People accept or ignore interruptions without knowing what each one will cost them, and the cost differs hugely between people and moments:

- **Small interruptions have large downstream costs.** A glance becomes an app, then a feed, then a slow return to the task (attention residue).
- **Notifications are treated alike.** A teammate’s question, a bank alert and a like arrive in the same tray with the same urgency.
- **Time is not attention.** An hour in an app can be deep work or passive drift; duration alone cannot tell them apart.
- **Everyone is different.** A like during deep work can cost one person 2 minutes and another 40.

3. Who faces it

What a “persona” means in Parallel

A persona is the twin’s starting guess about you. On day one the twin has no history, so it borrows the typical pattern of your kind of busy:

- when people like you usually focus best
- which apps are tools and which distract
- what mix of notifications arrives
- when fatigue tends to set in

Your own data (routine, check-ins, “did this matter?” taps, reality checks, real activity) then pulls the twin away from the persona and towards you. The personal model’s weight grows with evidence: n/(n+20). The four demo logins are one account per persona.

Persona	Starting focus peaks	Work tools	Most frequent pings	Fatigue usually from
Student	10 AM and 9:30 PM	Notion, Docs, lectures	Social likes, DMs (Instagram, Discord)	4 PM
Developer	11 AM and 11 PM	VS Code, Terminal, GitHub	Work chat (Slack), email, news	4:30 PM
Creator	2 PM and 8 PM	Figma, Docs, Notion	Social likes (their feed is also their work)	6 PM
Professional	9:30 AM and 3 PM	Excel, Docs, Notion	Work chat (Teams), email, meetings	3 PM

Who faces the problem

Persona	Typical day	What breaks focus	What they need
Students	Lectures, labs, self-study, exams	Instagram, YouTube, group chats during study blocks	Protection in exam weeks, without missing family or college alerts
Developers	Deep work, reviews, stand-ups	Slack pings mid-debug, news, video rabbit holes	Flow-state protection, with work chat batched
Creators	Editing, designing, posting	Their work tool (social media) is also their biggest distraction	“Create first, post later” boundaries
Professionals	Meetings, analysis, inbox	Email and chat volume, meeting fragmentation	Calm triage and protected analysis windows

4. Gaps in current solutions

Tool	Does well	What is missing
RescueTime, Rize	Track time, count context switches	Look backwards only; no prediction per notification
Freedom, Opal, ScreenZen	Block apps and sites on rules	Static rules, no learning, no “what if”
one sec	Adds friction before opening an app	Acts at app open, not at notification time; not personal
Forest	Motivation through gamification	No behavioural model at all
Apple Focus, Android Digital Wellbeing	Focus modes and summaries	Manual rules; no cost estimate, no simulation
Clockwise, Reclaim	Protect calendar focus time	No device or notification signal

The gap: no product predicts the downstream chain that one notification will cause for this specific person, shows the counterfactual, and then checks whether the prediction was right.

5. Our solution

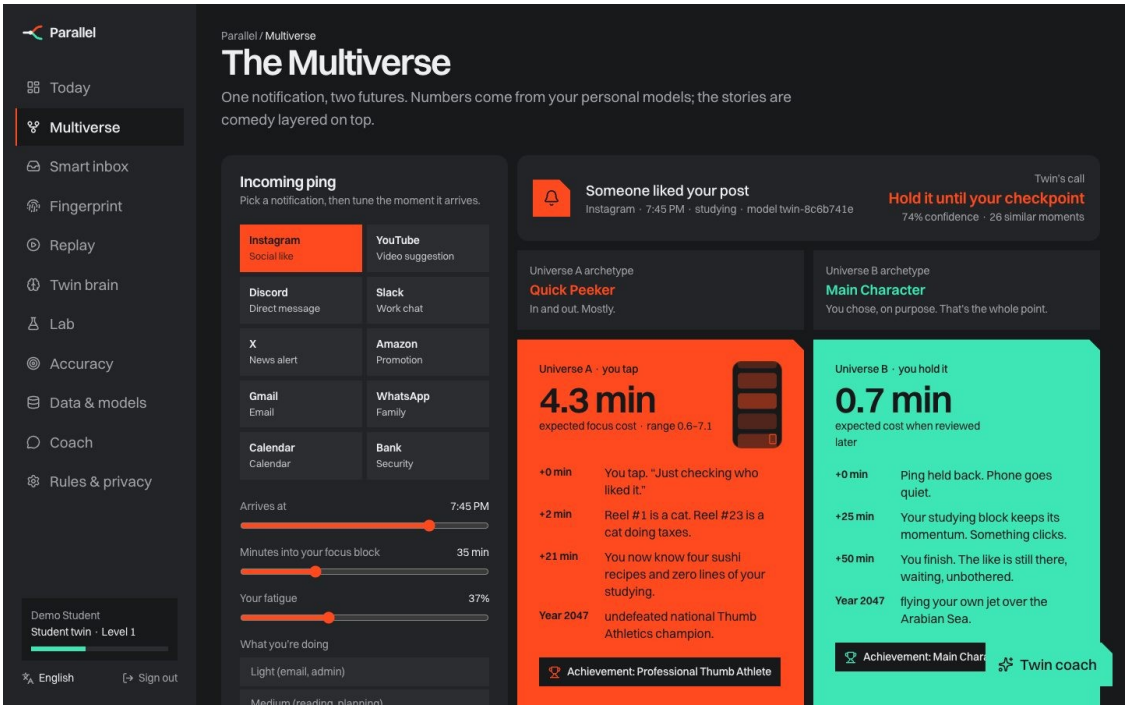
Parallel builds a twin of each user from three kinds of signal:

- what they declare: routine, weekly schedule, deadlines, which apps are tools and which nag
- what they tell it while working: 10-second energy check-ins, “did this matter?” taps, reality checks after a decision
- what it observes, with consent: app activity from ActivityWatch on desktop, and next, Android’s notification listener

Every notification is then run through the twin:

- the chance you open it
- the chance it becomes a rabbit hole
- the focus-minutes it is likely to cost (p10/p50/p90 from 2,000 simulated futures)
- the cost of holding it instead

The twin recommends allow, hold until your next checkpoint, or add to the digest. It explains why, names your class or work block if you are in one, and asks you when it is unsure.



The Multiverse: one notification, two futures. Numbers come from the model; the stories are a comedy layer.

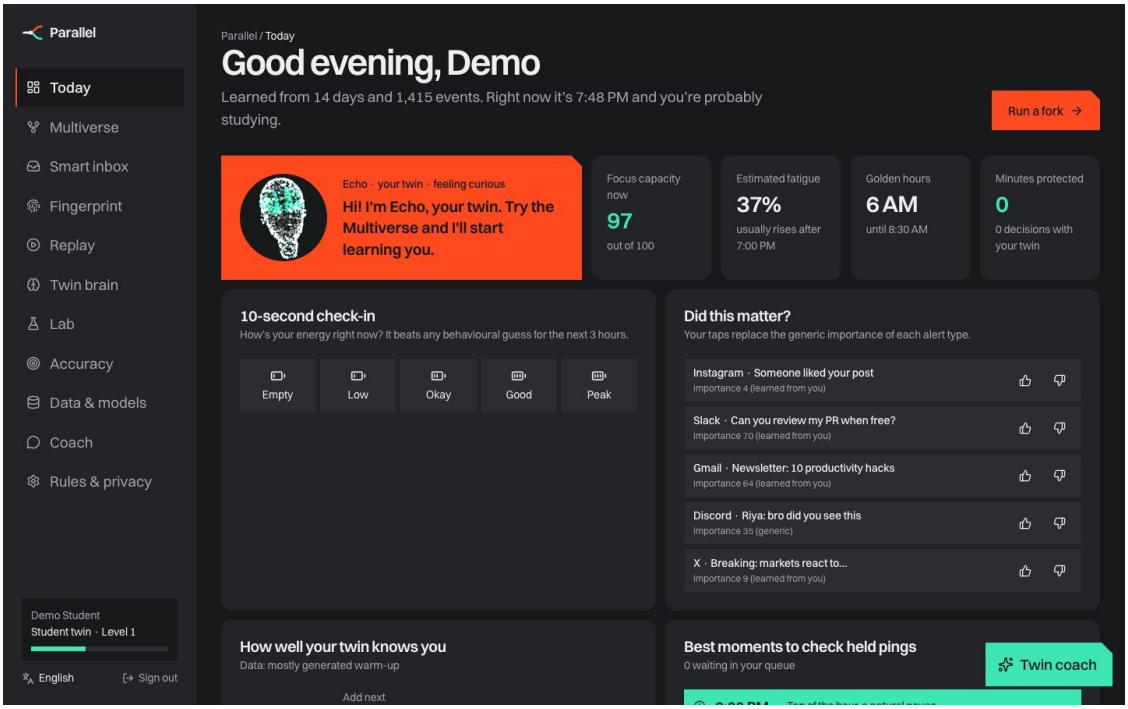
6. Our unique approach

1. **A counterfactual for every notification.** Allow, hold and digest are compared side by side, each with a range and a confidence.
2. **Causal, not just correlated.** A small randomized hold-back policy on low-importance alerts gives unbiased effects. Our T-learner matches those randomized estimates to within 0.9 minutes.
3. **Routine-aware.** Your declared day shifts the focus curve. Pings during classes or work blocks are held with the block named, and a close deadline makes the twin stricter.
4. **Personal with an honest cold start.** A population model trained on 60 users gives day-one predictions. They blend into your own model as evidence grows (personal weight $n/(n+20)$).
5. **Two layers: comedy over truth.** Universe A, "The Scrollfather", vs Universe B, "Main Character", makes the idea memorable. The minutes are never invented.
6. **Accuracy shown, not claimed.** Calibration, comparison with a generic rule, drift monitoring, data quality, a model registry and a tamper-evident audit chain.
7. **Your rules win.** Family, bank and calendar always pass. XP rewards intentional choices, not blanket blocking.
8. **Beyond a single app.** Time machine, attention doppelgänger, auto-reply drafts, focus reservation, an Attention API with an SDK, and privacy-safe team aggregates.

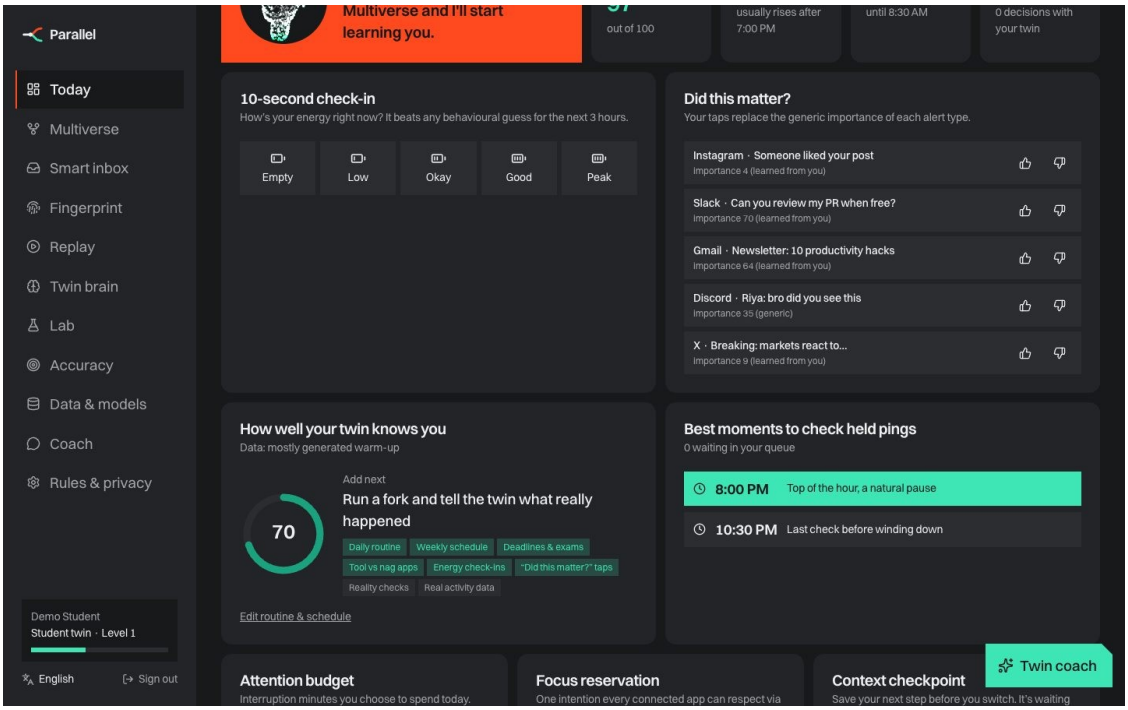
7. Key features (with prototype screenshots)

7.1 Today: state, inputs and recommendations

Current focus capacity and fatigue, golden hours, a 10-second energy check-in, "did this matter?" taps that re-weight each alert type, a meter showing how well the twin knows you, and the best moments to check held pings.



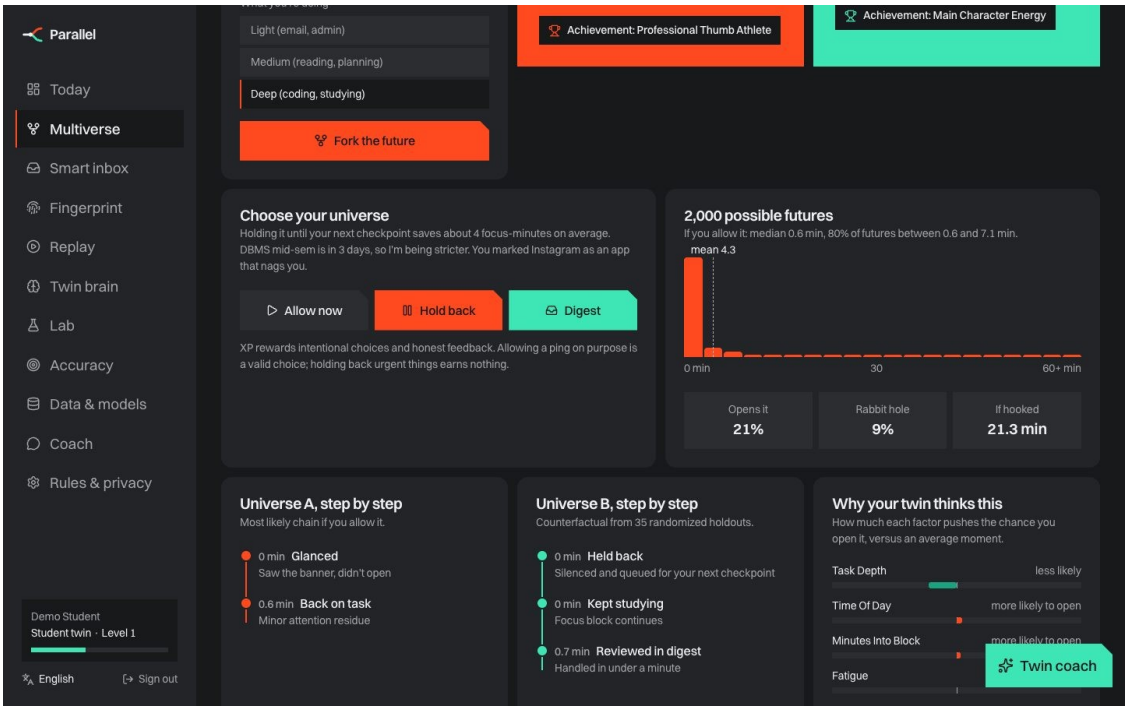
Today: Echo (the 3D twin companion), live state and the twin's learning inputs.



Ongoing inputs: energy check-in, "did this matter?", knowledge meter, check moments.

7.2 Multiverse simulator

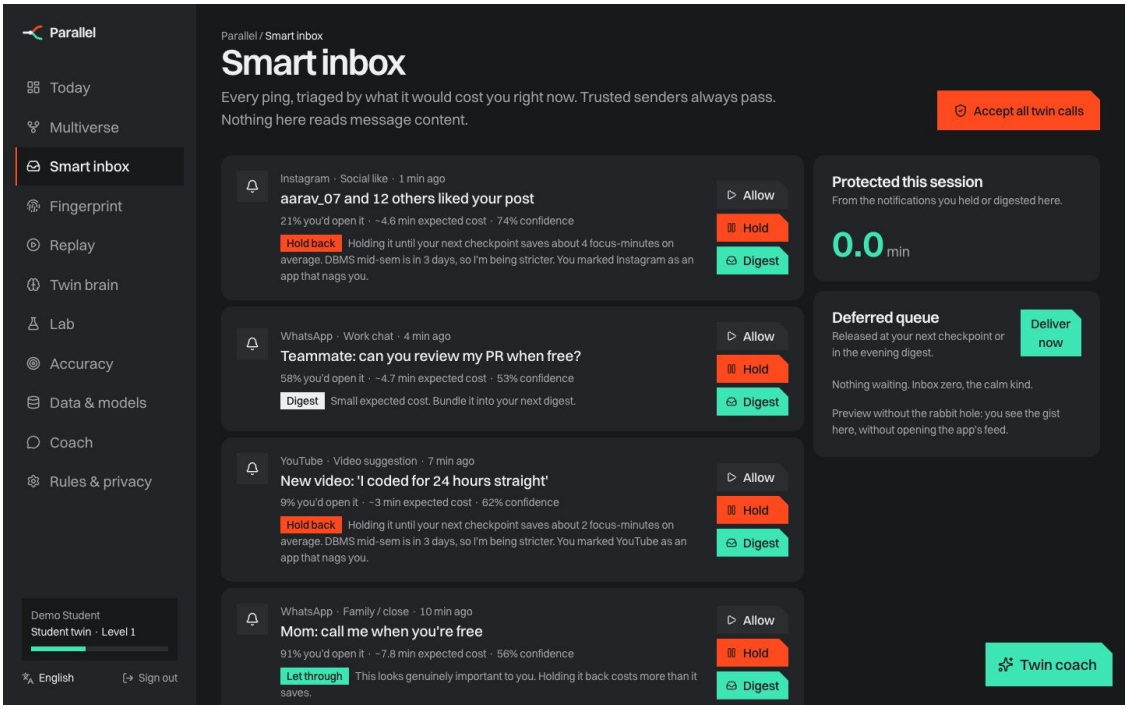
What-if sliders (time, task depth, fatigue, minutes into a block); two universes; the 2,000-future cost distribution; the most likely event chain in each universe; factor explanations; personal vs population vs blended prediction; then a reality check that grades the prediction.



Multiverse detail: decision, reality check, cost distribution and explanations.

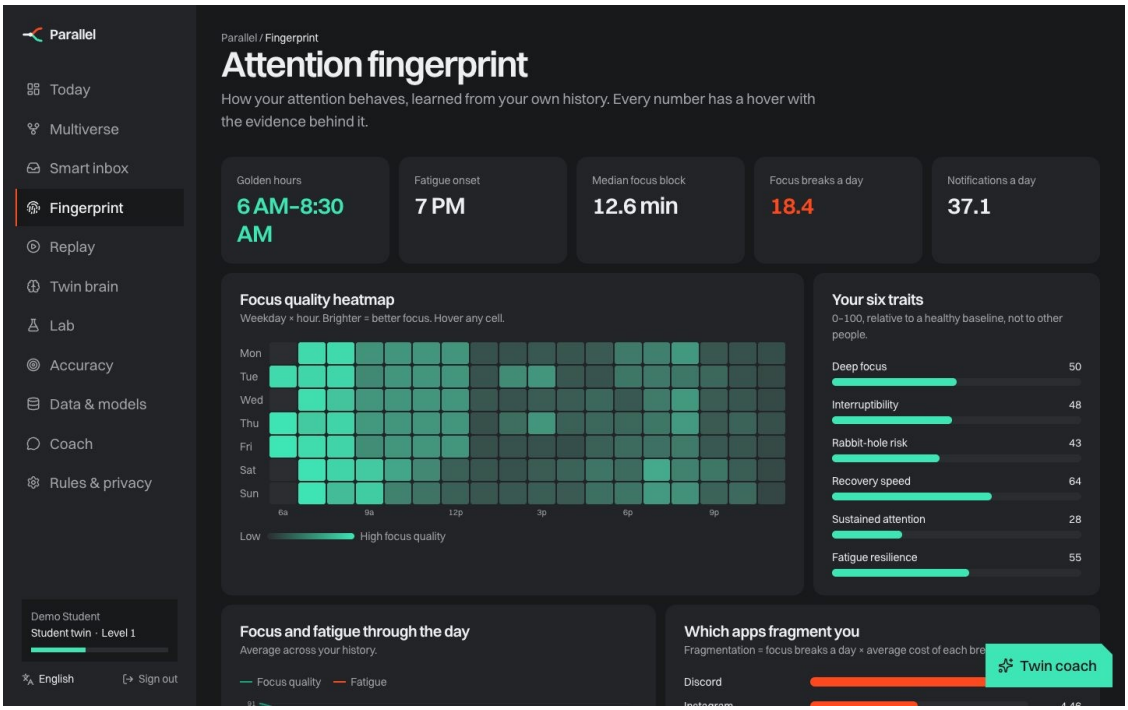
7.3 Smart inbox and auto-reply

Live triage with the twin's call for every ping, a deferred queue released at checkpoints or in the evening digest, and polite auto-reply drafts ("I'm in Lectures until 1:00 PM…") in four languages. Drafts are never sent automatically.

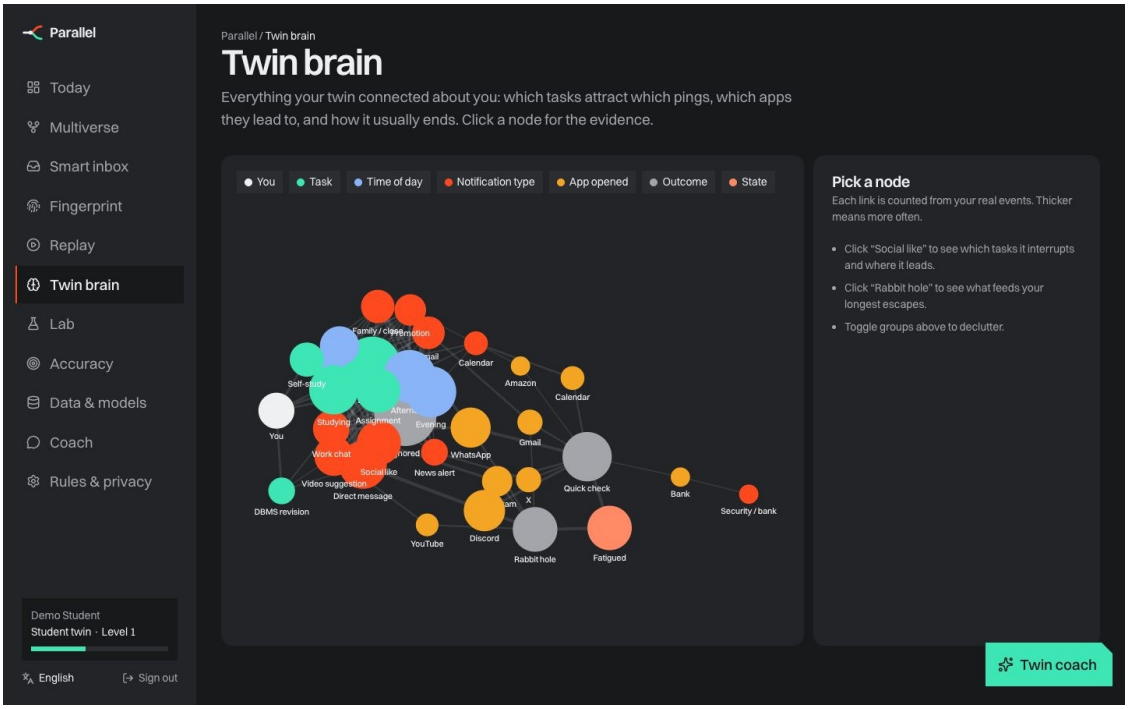


Smart inbox: predicted cost and recommendation for each notification.

7.4 Attention fingerprint and Twin brain

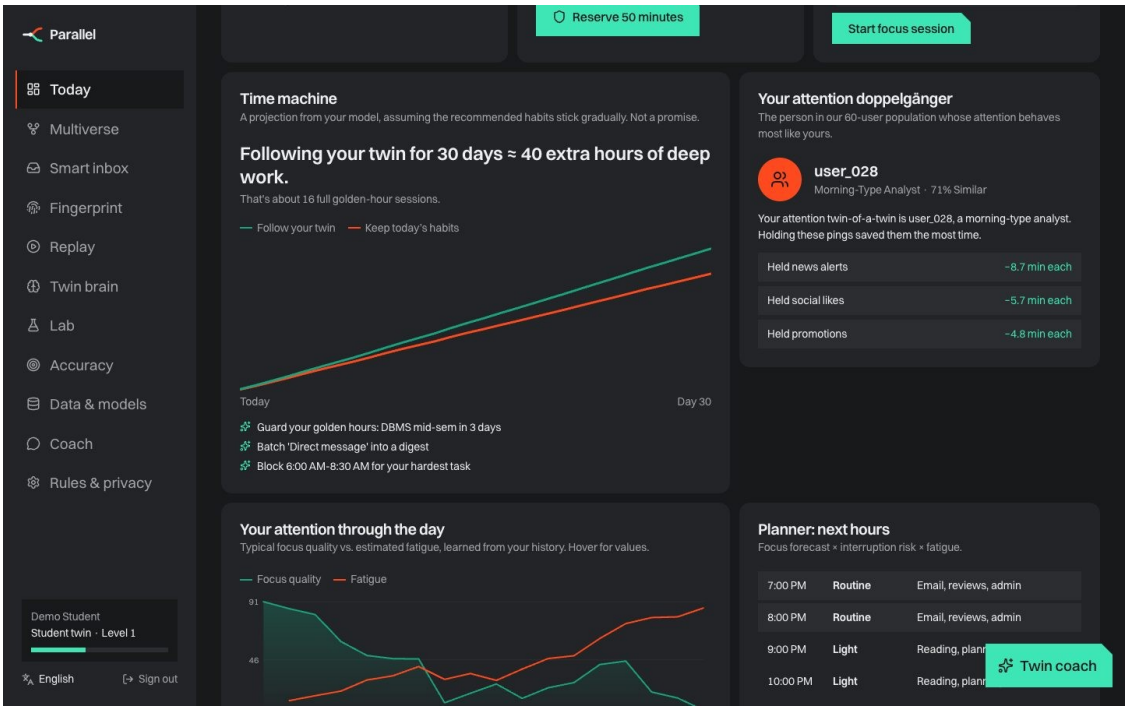


Fingerprint: weekday × hour focus heatmap, traits, day archetypes (KMeans) and randomized effects with 95% confidence intervals.



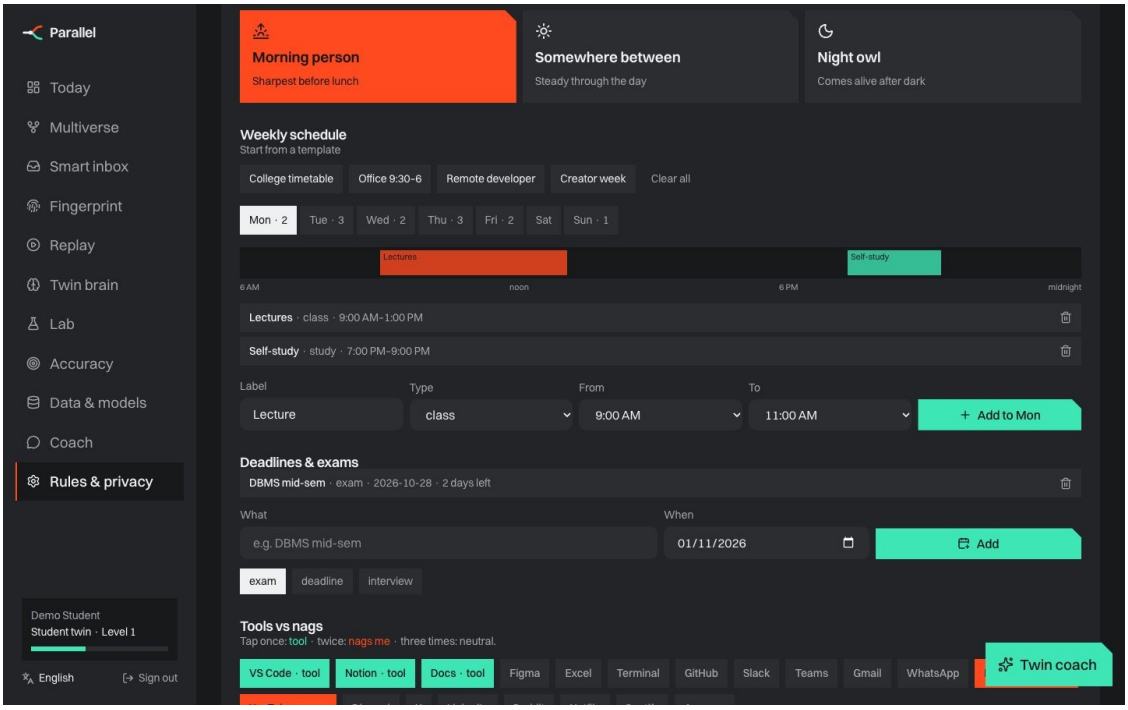
Twin brain: a knowledge graph of tasks, times, triggers, apps and outcomes, with evidence on click.

7.5 Time machine and doppelgänger



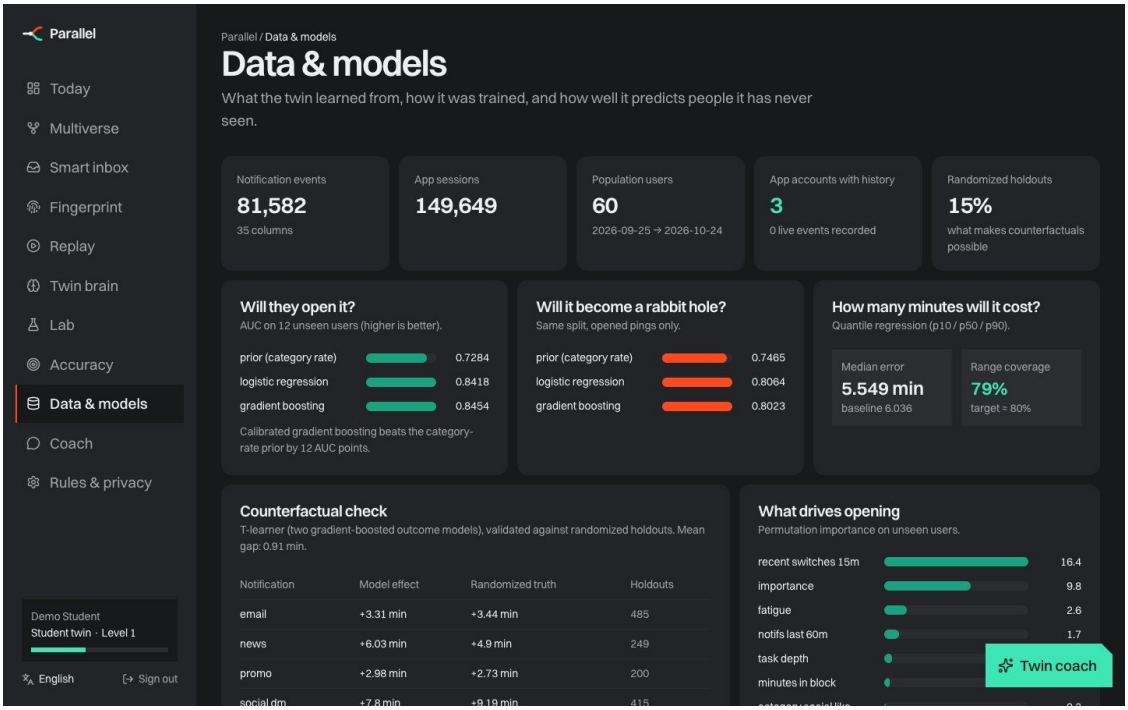
Thirty simulated days of current habits vs following the twin, and your closest behavioural match among 60 people.

7.6 Routine and schedule



Weekly schedule editor with templates, deadlines and exams, and tool vs nag apps (also part of the 7-step onboarding).

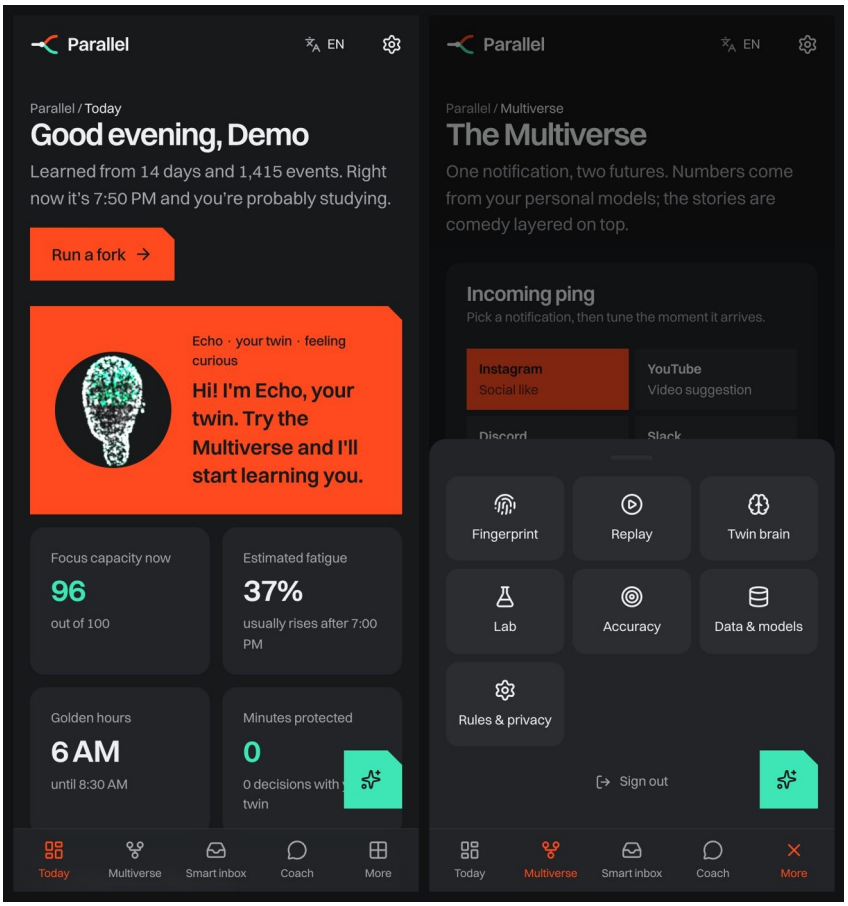
7.7 Data & models



Dataset, models against baselines on unseen users, counterfactual validation and the model registry.

7.8 Mobile app

The same codebase runs as an installable PWA and as native Android and iOS apps through Capacitor. Phones get a tab bar with a More sheet, safe-area layout, haptics and local reminders at check moments.

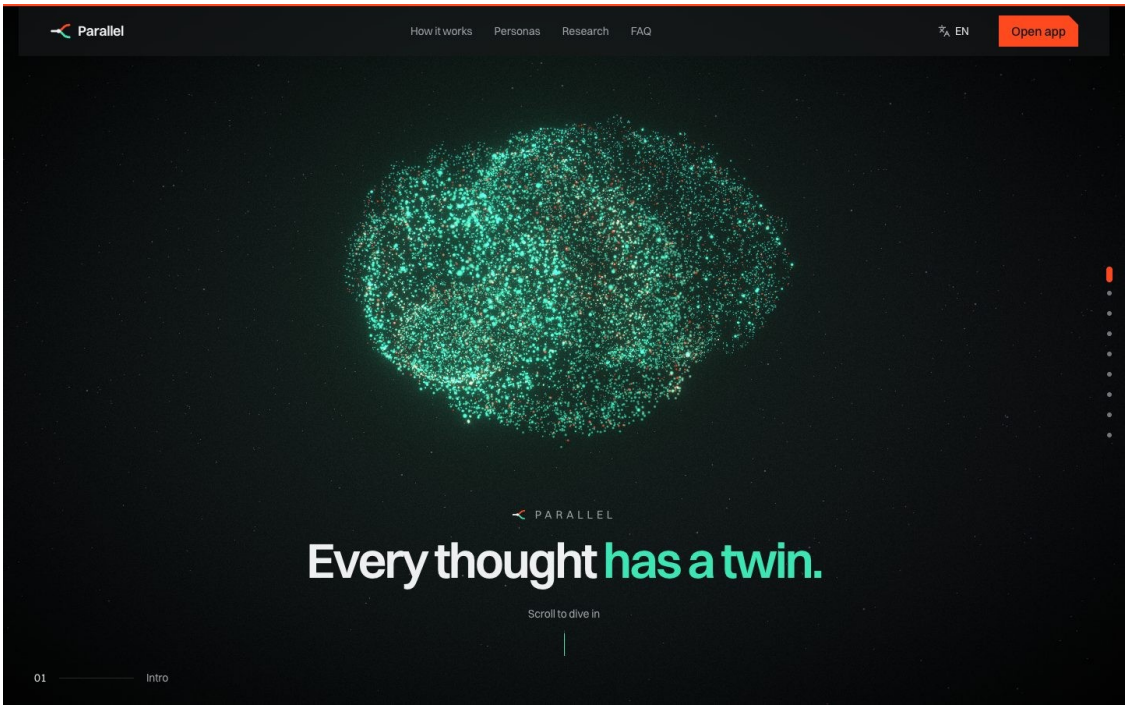


Phone layout: Today and the More sheet.

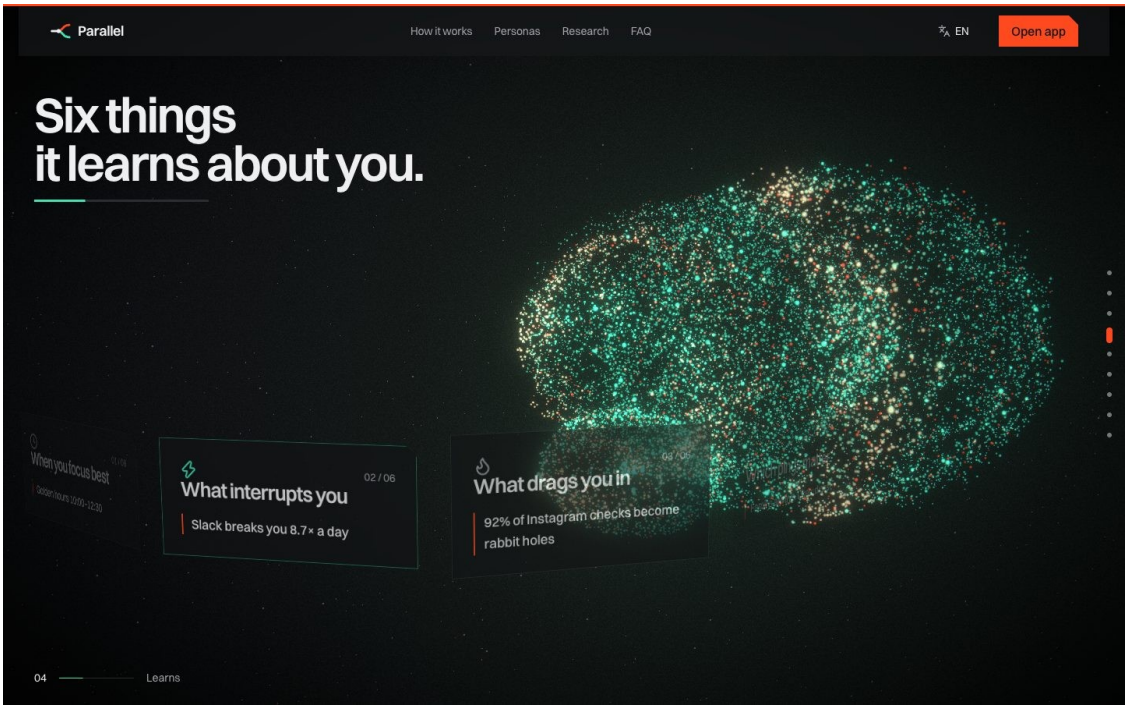
7.9 The 3D landing story

One persistent WebGL scene of about 15,000 GPU particles:

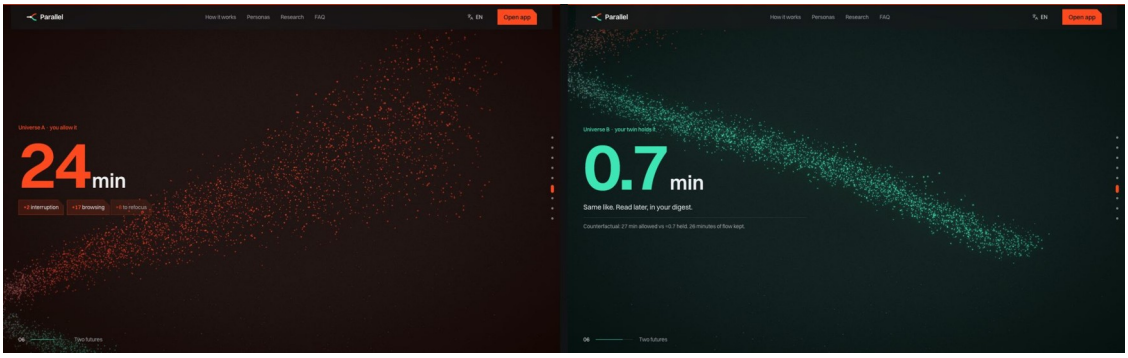
- the page opens on a living brain
- scrolling dives through it into the twin's hologram head
- then the six things the twin learns, on a rotating 3D cylinder
- the 7×24 attention fingerprint ring
- the notification fork, with a divider sweeping between Universe A (+2, +17, +8 = 27 min) and Universe B (0.7 min)
- the knowledge graph
- and finally you and your twin, face to face



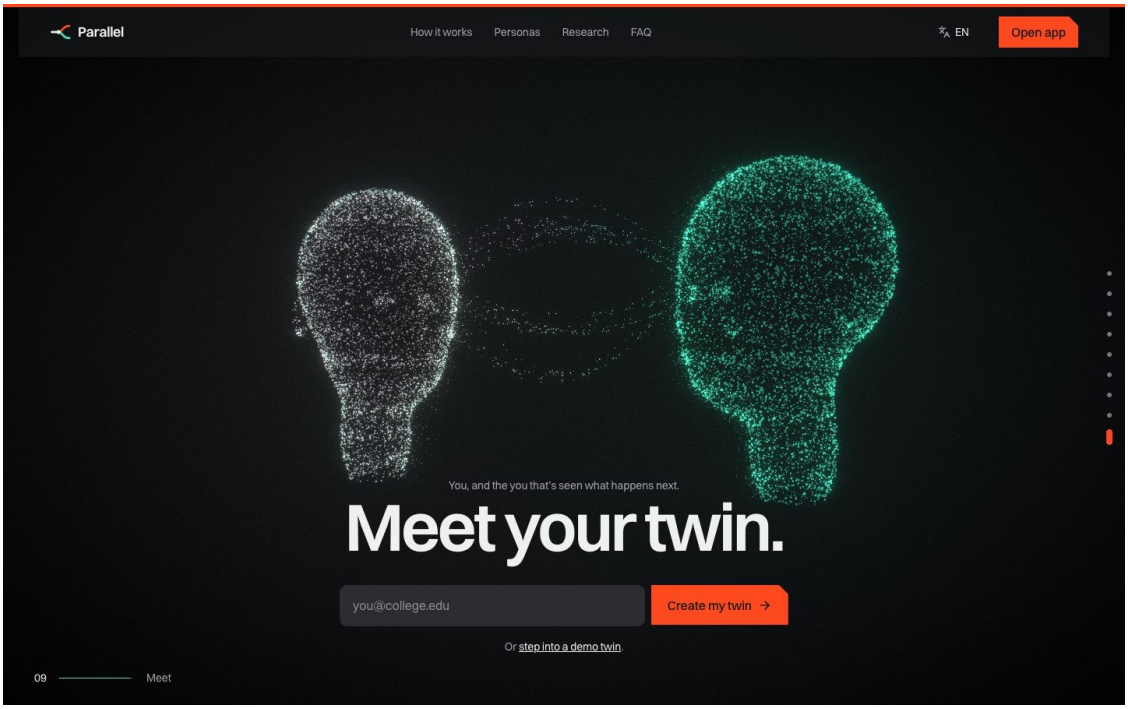
Opening: a living particle brain. Scrolling dives through it into the twin.



The six things the twin learns, on a rotating 3D cylinder over the brain.



The fork chapter: Universe A (allow) vs Universe B (hold), the problem statement's example.



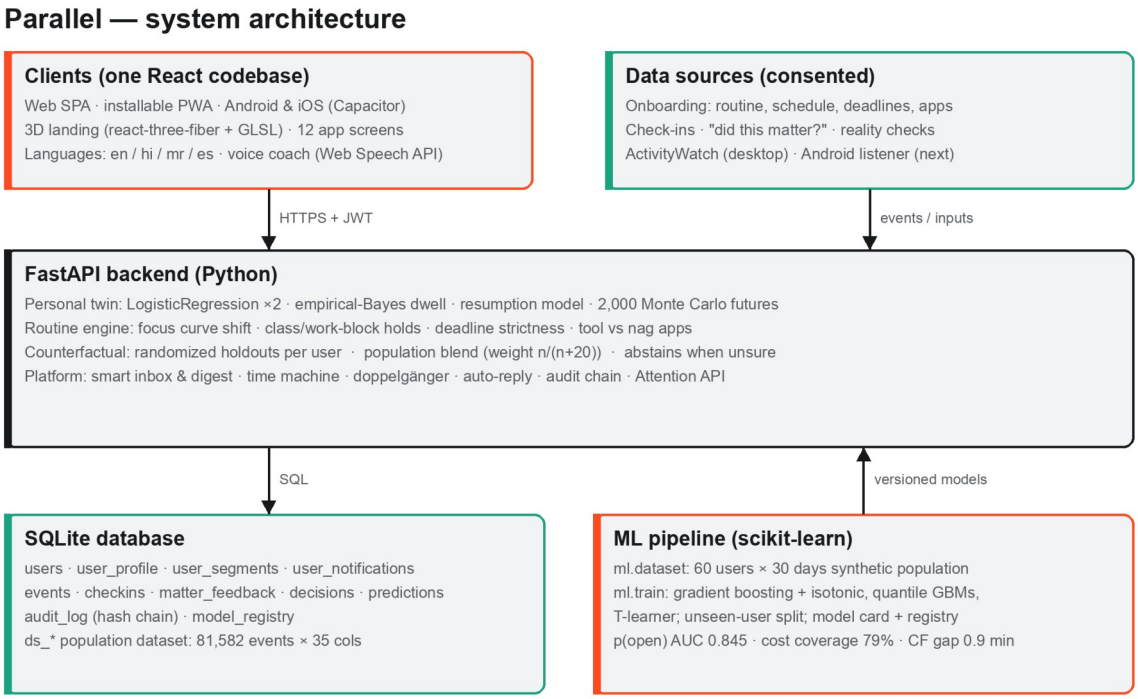
Finale: you and your twin, face to face.

8. Technology

Layer	Technology	Why
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4	Fast iteration, typed, one codebase for web and app
3D and motion	three.js, react-three-fiber, custom GLSL shaders, GSAP, Lenis, Motion	Cinematic scroll story with GPU particle morphing
UI components	React Bits (CountUp, Magnet, ClickSpark…), custom chart kit	Polished interactions; charts validated for colour-blind safety
Mobile	PWA (manifest + service worker), Capacitor 8 (Android/iOS)	Installable now; native store builds from the same code
Backend	Python, FastAPI, Pydantic, JWT, PBKDF2	Typed REST API; interactive docs at /docs

Layer	Technology	Why
Database	SQLite (PostgreSQL-ready schema)	Zero setup; real SQL; easy to inspect
ML	scikit-learn, NumPy, pandas, joblib	Calibrated, explainable models for small tabular data
Voice and languages	Web Speech API, i18next (en, hi, mr, es)	Free, on-device speech; inclusive for Indian users
Data source (desktop)	ActivityWatch (open source, local)	Real app-usage events with user consent

9. System architecture



Clients, data sources, the FastAPI backend, the SQLite database and the ML pipeline.

How one prediction flows:

1. Load the user's twin from their database rows, with their declared routine, learned importance and latest check-in.
2. Simulate 2,000 futures with the personal models.
3. Estimate the cost of holding the ping from randomized holdouts.
4. Apply rules and routine: trusted senders, class or work blocks, deadline strictness, nag apps, abstention when unsure.
5. Blend with the population model.

6. Log the prediction to the audit chain.
7. Later, grade the prediction against the user's reality check and refit the twin.

10. Use cases

Scenario	What Parallel does	Outcome
Student in DBMS exam week, 9 AM lecture, Instagram like arrives	Holds it: "You're in Lectures until 1:00 PM. DBMS mid-sem is in 3 days, so I'm being stricter." Offers check moments at 1:00 PM and 6:00 PM	No rabbit hole during class; the like is read in one batch
Developer debugging, teammate asks for a PR review	Work chat has high learned importance, so it lets it through; YouTube suggestions are held	Useful interruptions pass; low-value ones wait
Creator editing, DM from a friend	Holds it and drafts "I'm in a focus block until 3:00 PM…" in Hindi or English	The friend gets a polite reply; the edit continues
Professional at 4:30 PM with low energy (check-in 2/5)	Fatigue override; planner suggests a break; holds news and promos	Fewer late-day escapes when willpower is lowest
A third-party app asks "is now a good moment?"	Attention API returns now, defer or digest, with a reason and deliver_after	Apps respect one focus intention

11. Data inputs needed

Input	How it is collected	Why it is needed	Privacy
Routine (wake, bed, chronotype)	Onboarding / Settings	Shifts the focus curve and the active hours	Declared by the user
Weekly schedule	Onboarding editor with templates	Holds pings during classes and work blocks	Declared by the user
Deadlines and exams	Onboarding / Settings	Stricter holds and golden-hour protection	Declared by the user
Tool vs nag apps	Tap to classify	Classifies observed time; less benefit of the doubt for nags	App names only
Energy check-ins	10-second tap on Today	Overrides the behavioural fatigue estimate for 3 hours	A number from 1 to 5
"Did this matter?"	Thumbs up or down on recent pings	Learns importance per alert type	Category only
Decisions and	Multiverse, Smart	Grades predictions; trains	Minutes and

Input	How it is collected	Why it is needed	Privacy
reality checks	inbox	the twin	choices
App activity	ActivityWatch → /api/ingest/activitywatch	Real focus and distraction timeline	Window titles dropped; deduplicated
Notification metadata (next)	Android NotificationListenerService	Real notification events and holds	Package and category only; no text

Never collected: message text, notification bodies, window titles, keystrokes, screenshots, microphone audio or precise location.

12. Machine learning and evaluation

Population dataset (synthetic, clearly labelled):

- 60 users × 30 days
- 81,582 notification events × 35 columns
- 149,649 app sessions
- a 22% randomized hold-back policy on low-importance alerts

Models were evaluated on 12 users never seen in training (GroupShuffleSplit by user):

Target	Model	Result on unseen users	Baseline
p(open)	Gradient boosting + isotonic calibration	AUC 0.845, Brier 0.152	Category rate: AUC 0.728
p(rabbit hole | opened)	Gradient boosting + isotonic calibration	AUC 0.802	—
Focus cost p10/p50/p90	Quantile gradient boosting	MAE 5.5 min; 79% of outcomes inside the range	Category median: MAE 6.0
Counterfactual effect	T-learner	0.9 min mean gap vs randomized truth	—

Personal twins: two logistic regressions, empirical-Bayes dwell times, a resumption model, Monte Carlo rollouts and KMeans day archetypes. In backtests they score 10–24% better (lower Brier score) than a generic notification rule.

13. How to actually implement it

Phase	What ships	Status
1. Prototype	Twin, simulator, inbox, routine engine, dataset and models, web app, PWA	Done

Phase	What ships	Status
2. Real desktop data	ActivityWatch bridge script polling localhost:5600 and pushing to /api/ingest/activitywatch (tz_offset_min=330 for IST)	API ready; script next
3. Android	Capacitor plugin with NotificationListenerService: map package/category to a decision; cancel and re-post at check moments	Android project ready; plugin next
4. Calendar	Read-only .ics import to fill the weekly schedule	Planned
5. Scale	PostgreSQL + migrations, deployed API (Render/Fly) and web (Vercel/Netlify), monitoring	Planned
6. Personalization at scale	On-device updates, federated averaging with secure aggregation	Research

14. Feasibility

Dimension	Assessment
Technical	Proven: every component runs today. The models are light and train in under a minute on a laptop.
Platform	Android allows reading and holding notifications with user consent. Desktop works via ActivityWatch. iOS is limited to Focus filters, and the app reports this honestly.
Data	Cold start is solved by the population model plus declared routine. Personal accuracy improves with each reality check and live event.
Privacy	Metadata only; local-first design; export, portable profile, retention window, pause learning, one-click delete, hash-chained audit.
Cost	Runs on free tiers. No paid AI APIs; speech uses the browser engine.
Business	Consumer app plus an Attention API for other apps, and privacy-first team analytics (aggregates only, $k \geq 5$).

15. Risks and mitigations

Risk	Mitigation
Holding an important message	Trusted senders always pass; user rules override the model; the twin abstains when unsure; learned importance from “did this matter?”
Wrong predictions	Ranges and confidence shown; reality checks grade every prediction; calibration and drift monitoring

Risk	Mitigation
Over-optimizing focus	Rewards intentional choices (allowing a ping on purpose earns XP); a daily cap on hold-back XP; breaks encouraged
Feeling surveilled	Metadata only, local-first, transparent data page, delete anytime, no individual team scores
Synthetic-data bias	Clearly labelled; real ActivityWatch data replaces the warm-up; models retrain from live rows

16. Honest limits

- Day-0 history is still generated. It follows the user's declared routine, but it is not behaviour they actually showed. The Settings page says so, and real data replaces it as it arrives.
- Accuracy numbers are on synthetic data. The best demo upgrade is a few days of the presenter's own ActivityWatch data.
- Fatigue is a behavioural estimate, never a medical measurement.

17. Appendix: running the project

1. Clone github.com/SDP42/parallel.
2. Run ./run.sh on macOS/Linux, or run.ps1 in PowerShell on Windows.
3. Open http://localhost:5180 and sign in as student@parallel.app / parallel123 (also developer@, creator@ and professional@parallel.app).
4. For mobile: cd frontend && npm run android (or npm run ios on macOS). See docs/MOBILE.md.

More documentation in the repository: README, RUN, docs/ARCHITECTURE, docs/DATA_AND_ML, docs/MOBILE, docs/API, docs/FEATURES, docs/ROADMAP, docs/PRESENTER_GUIDE.